BYUI FTC Engineering Handbook

Version: 1.2 - Proposed

Created: September 25, 2026

Proposed Effective Date: October 1, 2026

Owner: BYUI FTC

Contents

1. Purpose and Scope
2. Roles and Responsibilities
3. Project Request and Approval
4. Repository Creation and Structure
5. Git Workflow
6. Documentation Standards
7. Environments and Deployment
8. Testing and Quality
9. Security and Access Management
10. Operations, Monitoring, Backups, and Incidents
11. Google Apps Script Projects
12. Team Member Departure Documentation
13. Review and Maintenance of This Handbook

Appendix: Project Proposal Template

Back to Contents

1. Purpose and Scope

This handbook defines the practices and standards used by the engineering team to plan, develop, document, test, deploy, and maintain systems for the BYUI FTC.

The goal is to make projects organized, secure, maintainable, and understandable by more than one person. These standards provide a common way of working without requiring every project to use the same technology.

These instructions apply to web applications, automations, Google Apps Script projects, internal services, APIs, data processes, reusable libraries, and other active engineering projects.

General Principles

- Every project must have an Owner, Author, Reviewer, and Tester.
- Every project must be registered in an official BYUI FTC repository.
- Important changes must be recorded and reviewed.
- Documentation must be kept in the official location defined for the project.
- Secrets and access must be managed securely.
- Deployments must be repeatable and reversible.
- Existing projects may adopt these standards gradually; new projects must follow them from the beginning.

2. Roles and Responsibilities

Each project must have the following four roles assigned. One person may hold more than one role when appropriate, but no role may remain unassigned.

Owner

The Owner of projects developed by this team is BYUI FTC. A Full-Time Employee participates in the approval consultation process when appropriate.

Author

The Author identifies a problem or opportunity, proposes a solution, and builds the project. Authors prepare proposals, implement approved solutions, follow this handbook, maintain the repository and documentation, and prepare the project for review, testing, and deployment.

Reviewer

The Reviewer examines the implementation before it is merged. The Reviewer checks alignment with the approved proposal, code quality, maintainability, possible bugs, security concerns, test results, documentation, and compatibility with existing systems. The Author should not be the only person approving their own changes.

Tester

The Tester validates the project in the appropriate environment, verifies the main functionality and relevant error cases, reports reproducible defects, confirms fixes, and provides testing approval before production deployment.

Project Approval

The Engineering Lead or Engineering Student Lead makes project approval decisions in consultation with the other engineering leads and a Full-Time Employee. A project must not begin implementation until it has been approved.

```
Problem Identified -> Consultation -> Proposal Submitted
-> Review and Consultation -> Decision
-> Implementation -> Review -> Testing -> Deployment
```

Back to Contents

3. Project Request and Approval

All new engineering projects must be proposed and approved before implementation begins. FTC employees identify problems, propose solutions, and present projects for approval.

The Author must first discuss the problem and proposed direction through an initial engineering consultation. After that consultation, the proposal must be submitted as a Project Proposal Issue in the central project proposal repository using the official template.

The proposal must explain the problem, proposed solution, intended audience, expected construction time, expected technology or platform, and expected result.

The Engineering Lead or Engineering Student Lead reviews the proposal with the other engineering leads and a Full-Time Employee. The decision must be recorded in the issue as Approved, Approved with Changes, Needs Clarification, or Not Approved. Only approved projects may proceed to implementation.

Significant changes to the project's purpose, proposed solution, audience, estimated construction time, or technical direction require another review.

Back to Contents

4. Repository Creation and Structure

A repository may be created only after the project proposal has been approved. Official project repositories must be created under the BYUI FTC GitHub organization, not in personal accounts. The repository must link to the approved Project Proposal Issue.

Repository names should be clear, descriptive, lowercase, use hyphens instead of spaces, and be consistent with the project name.

```
README.md
CONTRIBUTING.md
.env.example
docs/
.github/
  workflows/
  ISSUE_TEMPLATE/
  PULL_REQUEST_TEMPLATE.md
```

Each repository should use protected shared branches, required Pull Requests, reviewer approval, automated checks when available, separate Dev, optional Staging, and Production environments when appropriate, environment-specific secrets, automatic deletion of merged feature branches, and appropriate visibility and access permissions.

5. Git Workflow

The standard workflow uses short-lived feature or fix branches and Pull Requests. Direct pushes to shared branches are not permitted.

1. Start from the current **dev** branch.
2. Create a short-lived branch for the task.
3. Make focused commits.
4. Update the branch with the latest **dev** changes before opening or updating the Pull Request.
5. Resolve conflicts when necessary.
6. Open a Pull Request into **dev**.
7. Complete review and automated checks.
8. Merge the Pull Request into **dev**.
9. Validate the change in Dev.
10. If Staging is used, promote the approved version to Staging and validate it.
11. Promote the approved version to Production.

Recommended branch names are **feature/<short-description>**, **fix/<short-description>**, and **chore/<short-description>**.

5.2 Conventional Commits

Commit messages must follow the Conventional Commits format:

```
<type>(optional scope): <description>
```

Examples:

```
feat(auth): add Google sign-in
fix(queue): prevent duplicate check-ins
docs(readme): update deployment instructions
```

Recommended commit types are:

- **feat**: New functionality.
- **fix**: Bug correction.
- **docs**: Documentation changes.
- **style**: Formatting changes that do not affect logic.
- **refactor**: Code restructuring without behavior changes.
- **test**: Adding or modifying tests.
- **chore**: Maintenance tasks.
- **build**: Build system or dependency changes.
- **ci**: CI/CD changes.
- **perf**: Performance improvements.
- **revert**: Reverting a previous commit.

Commit descriptions should be lowercase, use imperative language, be concise, and not end with a period.

5.3 Semantic Versioning

Project releases must use Semantic Versioning:

```
MAJOR.MINOR.PATCH
```

- MAJOR: Breaking changes.
- MINOR: New backward-compatible functionality.
- PATCH: Backward-compatible bug fixes or small corrections.

Releases must be connected to Git tags using the following format:

```
v1.0.0
v1.1.0
v1.1.1
```

Conventional Commits describe individual changes. Semantic Versioning describes project releases.

5.4 Naming Conventions

The following naming conventions must be used unless a technology requires a different standard:

Item	Convention	Example
Repository names	Lowercase kebab-case	**student-dashboard**
Branch names	Lowercase with a type prefix	**feature/add-login**
Git tags	Lowercase **v** plus Semantic Version	**v1.2.0**
JavaScript variables	camelCase	**userName**
JavaScript functions	camelCase	**getUserData()**
Classes and components	PascalCase	**UserDashboard**
Constants	UPPER_SNAKE_CASE	**MAX_RETRIES**
Environment variables	UPPER_SNAKE_CASE	**DATABASE_URL**
Folders and files	Lowercase kebab-case	**user-service.js**
Google Apps Script functions	camelCase	**sendWeeklyReport()**

The handbook uses consistent capitalization for official terms such as **Local**, **Dev**, **Production**, **Pull Request**, **Project Proposal**, **Engineering Lead**, **Owner**, **Author**, **Reviewer**, and **Tester**.

Feature and fix branches should be deleted after their Pull Requests are merged. Deleting a merged branch does not delete its commits or history. Permanent shared branches such as **dev** and **main** must not be deleted.

Use a Revert Pull Request to undo a change while preserving history. To return to an exact earlier release, deploy the commit or tag associated with the known stable version. Do not use **reset --hard** or force-push on shared branches.

5.5 Branch Protection and Repository Rules

The BYUI FTC GitHub organization uses GitHub Team. Private repositories must use GitHub branch protection rules or rulesets to enforce the standard development workflow.

The **dev**, **stage**, and **main** branches must be protected with the following requirements:

- Changes must be submitted through a Pull Request.
- At least one approval from the assigned Reviewer is required before merging.
- Required automated checks must pass before merging.
- Conversations and requested changes must be resolved before merging.
- Direct pushes must be restricted.
- Force pushes must be blocked.
- Branch deletion must be blocked.
- Administrators and other users must not bypass these rules except when an authorized emergency correction is required.

GitHub Actions should provide the required status checks, including automated validation, testing, and other checks appropriate to the project. Local Git hooks may be used as an additional safeguard, but they do not replace GitHub branch protection because they can be bypassed or may not be installed on every contributor's machine.

When possible, organization-level rulesets should apply these requirements automatically to new and existing repositories. Repository templates and automation should also be configured so that new repositories receive the expected workflows and protection settings.

5.5.1 Local Git Hooks

For security purposes and to safeguard the integrity of the BYUI FTC repositories, projects should implement and document local Git hooks as an additional layer of protection. Local hooks provide immediate feedback and help prevent common mistakes before changes are pushed to GitHub.

When appropriate, local hooks should:

- Prevent direct pushes to protected branches such as **dev**, **stage**, and **main**.
- Validate Conventional Commit messages.
- Run formatting and linting checks.
- Run quick tests before code is pushed.
- Identify common configuration or security problems before they reach the repository.

The required hooks and their setup instructions must be documented in the repository. A shared tool or repository-managed configuration should be used when practical so that contributors follow the same validation rules.

Local Git hooks do not replace GitHub branch protection, required Pull Request reviews, or automated status checks. Because local hooks can be bypassed or may not be installed on every contributor's machine, GitHub remains the authoritative enforcement layer.

5.6 GitHub Organization and Repository Access

GitHub access must be managed separately from the project roles of Owner, Author, Reviewer, and Tester. The project Owner is always BYUI FTC. GitHub roles determine what a person can do within the organization or a repository.

Organization Owner

An Organization Owner has full administrative access to the BYUI FTC GitHub organization and its repositories.

Only Organization Owners may:

- Add, remove, or promote Organization Owners.
- Manage organization-wide settings, members, teams, and billing.
- Approve the assignment or removal of Repository Administrators.

Organization Owner access must be limited to authorized Full-Time Employees or other institutionally authorized administrators.

Repository Administrators

Repository Administrators must be managed through an organization team such as **repository-administrators**. When appropriate, this team should receive the All-repository admin role.

Repository Administrators may manage assigned repositories, including:

- Repository settings and access.
- Branch protection rules and rulesets.
- GitHub Actions workflows, secrets, variables, and environments.
- Deployments and repository configuration.
- Repository deletion when authorized.

Repository Administrators cannot add, remove, or promote Organization Owners. They must not assign Repository Administrator access to another person without approval from an Organization Owner.

Collaborators

Collaborators should receive **Write** access only to the repositories assigned to them. They may create branches, push code, create Issues, open Pull Requests, review changes, and merge Pull Requests when all required rules and checks have passed.

Collaborators must not be able to delete repositories or delete or force-push protected branches such as **main**, **stage**, and **dev**. These restrictions must be enforced through repository permissions and branch protection rules.

Reviewers

Reviewers should receive **Write** access because GitHub requires that level for approving and merging Pull Requests. GitHub Team does not provide a built-in role that allows a person to approve and merge Pull Requests while completely removing their ability to push to unprotected branches.

Reviewers must not receive **Admin** access. Direct pushes to protected branches must be blocked, and Pull Requests must require the appropriate reviews and automated checks before merging.

Testers

Testers should receive **Triage** access by default. This allows them to view repositories, manage Issues and Pull Requests, and record testing results without pushing code. A Tester may receive **Write** access only when the project requires the Tester to modify test code or configuration.

Access Management Principles

- Access should be granted through teams whenever possible.
- Users should receive the lowest repository role that allows them to perform their responsibilities.
- Organization Owner and Repository Administrator access must be reviewed when a person changes responsibilities or leaves the FTC.
- Ruleset bypass permissions must be limited and documented.
- Administrative actions and access changes should be recorded in the organization or repository audit log when available.

Back to Contents

6. Documentation Standards

Documentation must be clear, current, and stored in one official location. Information should not be split across multiple places without links between them.

Documentation	Official location	Purpose
Project overview and quick start	**README.md**	Explains what the project is and how to begin.
Setup, deployment, testing, and architecture	**docs/** or Wiki	Provides detailed technical instructions.
Troubleshooting and user guidance	Wiki or **docs/**	Records recurring problems and procedures.
Technical decisions	**docs/decisions/**	Explains important decisions and context.
Project proposal	Proposal repository	Records why the project was proposed and approved.
Team-wide rules	Engineering Handbook	Defines standards for all projects.
Tasks and defects	GitHub Issues	Tracks work that needs to be completed.

The README must include the project name, purpose, audience, technologies, dependencies, local setup, configuration, testing, deployment, Dev, Staging when applicable, and Production links, Owner, Author(s), Reviewer, Tester, and the approved proposal link.

This requirement applies to all applications, regardless of where they are hosted or executed, including Google Apps Script, **content.byui.edu**, local servers, cloud platforms, and other approved environments. Regardless of the hosting environment, all source code, documentation, configuration examples, and deployment instructions must be maintained in the project's official GitHub repository.

The Wiki should be used for detailed, long-form documentation such as operational procedures, troubleshooting, user instructions, and frequently asked questions. The README remains the primary entry point.

If a change affects installation, configuration, testing, deployment, architecture, or system behavior, the related documentation must be updated in the same Pull Request.

Back to Contents

7. Environments and Deployment

The standard environment model is Local, Dev, optional Staging, and Production. Staging should be added when the project's size, risk, or testing needs justify a separate pre-production validation environment.

Environment	Purpose
Local	Individual development and early testing on the Author's machine.
Dev	Shared integration environment where merged changes are deployed and validated.
Staging (optional)	Pre-production validation for projects that require a closer production replica.
Production	Live environment used by real users. Access and deployments must be restricted.

Changes merged into **dev** should deploy automatically to Dev when CI/CD is configured. When Staging is used, the approved version should be promoted there for pre-production validation before Production. Production deployments should initially be manual or approval-controlled. Every deployment should be associated with a commit, tag, or release so the exact deployed version can be identified and restored.

GitHub Actions is the default option for automated checks and deployments. The deployment process must be documented in the repository.

Back to Contents

8. Testing and Quality

Testing must be proportional to the project's size, complexity, and importance. Every project must have a Tester and define what will be checked before production deployment.

At minimum, the project must run locally, the main workflow must work as intended, relevant error cases must be tested, the change must be reviewed by someone other than the Author, automated checks should run when practical, and testing results and known limitations must be recorded in the Pull Request.

The Tester must confirm that the approved functionality works in Dev and, when applicable, Staging before promotion to Production. Defects should include steps to reproduce, expected behavior, and actual behavior.

A task is complete when the implementation is merged, required documentation is updated, review is complete, testing is complete, and the change has been deployed to the required environment.

9. Security and Access Management

- Never commit passwords, API keys, tokens, or other secrets to a repository.
- Use GitHub Secrets, environment secrets, or an approved secret manager.
- Use separate credentials for Local, Dev, Staging when applicable, and Production when possible.
- Give users only the access required for their responsibilities.
- Protect production branches and deployment environments.
- Use multi-factor authentication for GitHub and related services.
- Review access when a person changes responsibilities or leaves the team.
- Document sensitive data and relevant security requirements.

Suspected exposed credentials, unauthorized access, or serious vulnerabilities must be reported to the Engineering Lead promptly. Exposed credentials must be revoked and replaced, not merely removed from the latest commit.

10. Operations, Monitoring, Backups, and Incidents

For important systems, documentation should identify where the application runs, where logs can be viewed, what indicates a failure, who should be contacted, how to redeploy or roll back, what data is backed up, how restoration works, and any scheduled jobs or external dependencies.

When a production issue occurs, the team should protect users and data, identify the problem, communicate with relevant stakeholders, restore service, and document the cause and corrective action.

Significant incidents should result in a short written record describing what happened, the impact, the resolution, and actions needed to prevent recurrence.

Projects should be periodically classified as Active, Maintenance Only, Candidate for Migration, or Candidate for Retirement. Retired systems must be archived with final documentation and reviewed access.

11. Google Apps Script Projects

11.1 Purpose

Google Apps Script projects require additional standards because their code is stored in GitHub while execution, permissions, deployments, versions, triggers, and integrations are managed through Google Apps Script.

This section applies to Apps Script automations, web apps, Google Workspace add-ons, API executables, and scripts connected to Google Sheets, Docs, Drive, Gmail, or other Google services.

11.2 Source Control

All Apps Script projects must be connected to an official GitHub repository.

The repository must contain the complete source code, including:

- **.js**, **.gs**, **.html**, or related source files.
- **appsscript.json**.
- Setup and deployment documentation.
- Configuration examples.
- Testing instructions.
- A list of required Google services and permissions.

Where practical, the team should use **clasp** to synchronize the Apps Script project with the GitHub repository.

Changes should be made through the standard Git workflow and must not exist only in the Apps Script editor.

11.3 Project Ownership

Apps Script projects and production deployments should be associated with an organizational account controlled by the BYUI FTC whenever possible.

Personal accounts should not be the only owners of critical production scripts. Ownership and deployment access must be documented in the project README.

11.4 Environments

Apps Script projects should use the following environment model:

- Local: The Author's local development and code-validation environment.
- Dev: The shared Apps Script project or deployment used for integration and testing.
- Staging (optional): The Apps Script project or deployment used for pre-production validation when the project requires it.
- Production: The Apps Script deployment used by real users or business processes.

For important or high-impact systems, Dev, Staging when applicable, and Production should use separate Apps Script projects. This prevents development code, triggers, permissions, and test data from affecting Production.

11.5 Versions and Deployments

Apps Script versions and deployments must be treated as separate concepts.

- A version is an immutable snapshot of the script code.
- A deployment makes a specific version available to users or other systems.

The Dev environment may use a head deployment for testing. Head deployments must not be used for public or production use.

Production must use a versioned deployment. Each production release must include a version number, meaningful deployment description, related Git commit, Pull Request, or release tag, deployment date, and the person who approved or performed the deployment.

To update an existing production deployment, create a new Apps Script version and update the deployment to use that version. The existing deployment URL or ID should be preserved when possible.

11.6 Configuration and Script Properties

Environment-specific values must not be hardcoded in source code. Examples include Spreadsheet IDs, Folder IDs, API endpoints, email addresses, feature configuration, and external service credentials.

Use separate configuration values for Dev, Staging when applicable, and Production. Sensitive values must be stored using an approved secure method and must not be committed to GitHub. The repository should show which values are required without including real secrets.

11.7 Triggers

Triggers must be documented and reviewed. Documentation must identify the trigger type, function, schedule or event, account used to run it, environment, required permissions, and expected behavior if it fails.

Triggers must be configured separately for Dev, Staging when applicable, and Production. An Author must not create or modify Production triggers without authorization.

11.8 Permissions, OAuth Scopes, and Institutional Accounts

The project must request only the Google services and OAuth scopes it requires.

Apps Script projects must be created, authorized, and deployed using BYU-Idaho Google Workspace accounts, under the **@byui.edu** domain.

Personal Gmail accounts must not be used as the only owner, authorization account, trigger owner, or production deployment account for official BYUI FTC projects.

When an OAuth consent screen or application audience must be configured, the project should use the BYU-Idaho Google Workspace organization and the approved institutional domain.

Before Production deployment, the Reviewer must verify:

- Required permissions.
- Authorized users.
- Access to connected files, folders, spreadsheets, or services.
- The account used to execute the script.
- Whether the script handles sensitive information.
- That the project is using an approved BYU-Idaho institutional account.

Changes that add new Google services or permissions must be reviewed before deployment.

11.9 Testing

Apps Script testing should include local code validation when practical, testing in Dev, representative non-production data, time-based and event-based triggers, permissions and authorization behavior, expected error cases, and relevant quotas and execution limits.

The Tester must record the result in the Pull Request or related project issue.

11.10 Deployment Process

An Apps Script deployment may proceed only after the change has been implemented in the Author's branch, the Pull Request has been reviewed, required checks have passed, the change has been deployed and tested in Dev, the Tester has approved the result, and the production version and deployment details have been documented.

The Production deployment must use the approved version of the script.

11.11 Rollback and Recovery

If a production deployment causes a problem, identify the last known stable version. The preferred recovery options are to update the existing deployment to the previous stable version, revert the related Git change and create a new version, or redeploy the last known stable version.

The team must not rely only on the Apps Script editor's project history. The GitHub repository is the primary source of code history.

11.12 Required README Information

Every Apps Script repository must document the purpose of the script, connected Google services, Apps Script project ID when appropriate, Dev, Staging when applicable, and Production deployment IDs or URLs, required Script Properties, required triggers, required permissions and OAuth scopes, local setup instructions, deployment instructions, Owner, Author, Reviewer, Tester, and the approved Project Proposal Issue.

Google Apps Script versions are immutable snapshots, and versioned deployments allow a specific version to remain in use while new changes are developed. Google Apps Script deployment documentation Google Apps Script versions documentation

12. Team Member Departure Documentation

12.1 Purpose

This section defines the documentation required when an Author, Reviewer, or Tester leaves the FTC or is no longer responsible for a project.

The goal is to ensure that project knowledge, access information, and operational details remain available to the team.

12.2 Departure Documentation Page

Before leaving the FTC, the team member must create or update a page in the project repository documenting the transition.

The page should include:

- Current project status.
- Outstanding tasks and known issues.
- Important technical decisions.
- Local setup instructions.
- Deployment instructions.
- Testing instructions.
- Relevant dependencies and integrations.
- Apps Script triggers, deployments, and permissions, when applicable.
- Known risks or limitations.
- Recommended next steps.

The page should be stored in the repository's **docs/** directory or project Wiki and linked from the README.

12.3 Notification

After the documentation page has been created or updated, the departing team member must notify:

- The Engineering Lead or Engineering Student Lead.
- A Full-Time Employee.
- The project's current Author, Reviewer, or Tester, when applicable.

The notification should include a link to the repository page.

12.4 Access and Responsibilities

The project team must review the departing member's responsibilities and access, including:

- GitHub repository access.
- Google Apps Script access.
- Google Workspace files and folders.
- Deployment permissions.
- Secrets and environment variables.

- Production systems.
- External services and APIs.

Access that is no longer required must be removed, and replacement responsibilities must be documented in the README.

12.5 Completion

The departure documentation is complete when:

- The repository page has been created or updated.
- The page is linked from the README.
- The Engineering Lead or Engineering Student Lead and a Full-Time Employee have been notified.
- Relevant access has been reviewed.
- Open work has been transferred to GitHub Issues.

Back to Contents

13. Review and Maintenance of This Handbook

This handbook should be reviewed periodically by the Engineering Lead and the engineering leads.

Updates should be made when a standard changes, a new platform or workflow is adopted, a repeated problem reveals that an instruction is unclear, or a security, deployment, or operational practice needs to change.

Changes to this handbook should be reviewed through a Pull Request in the handbook repository.

Appendix: Project Proposal Template

Use this template for Project Proposal Issues in the central proposal repository.

Project Information

Project Name

<!-- Enter the proposed name of the project. -->

Request Date

<!-- Enter the date when this proposal is submitted. -->

Author(s)

<!-- List the person or people proposing and building the project. -->

Owner

BYUI FTC

Problem

What problem has been identified?

<!-- Clearly describe the problem or opportunity. -->

Who is affected by this problem?

<!-- Identify the people, teams, or processes affected. -->

How is the problem currently handled?

<!-- Describe the current process, workaround, or limitation. -->

Why should this problem be addressed now?

<!-- Explain the importance, urgency, or expected benefit. -->

Purpose

What is the purpose of this project?

<!-- Describe the main purpose of the proposed project. -->

What outcome should the project produce?

<!-- Describe the expected result after the project is completed. -->

What would success look like?

<!-- Explain how the team will determine whether the project achieved its purpose. -->

Audience

Who will use the project?

<!-- Identify the intended users. -->

How many users are expected?

<!-- Provide an estimate if possible. -->

Are the users internal, external, or both?

<!-- Describe the type of users. -->

Proposed Solution

What solution is being proposed?

<!-- Describe the proposed solution at a high level. -->

How will the proposed solution address the problem?

<!-- Explain how the solution will improve the current situation. -->

What are the main functions of the proposed solution?

<!-- Describe the primary capabilities. -->

Technical Information

Proposed Technology or Platform

<!-- Identify the expected programming language, framework, service, or platform. -->

Expected Integrations or Dependencies

<!-- List external systems, APIs, services, or libraries that may be required. -->

Expected Data Sources

<!-- Describe where the project's data will come from. -->

Expected Hosting Environment

<!-- Indicate where the project is expected to run. -->

Database Requirement

<!-- Indicate whether the project requires a database. If yes, provide details. -->

Estimated Construction Time

Estimated Development Time

<!-- Provide an estimated duration or range. -->

Target Completion Date

<!-- Provide a target date if one has been identified. -->

Additional Notes

<!-- Include any relevant information about the estimate. -->

Approval

This section must be completed by the Engineering Lead or Engineering Student Lead after consultation with the other engineering leads and a Full-Time Employee.

Decision

<!-- Select one: Approved / Approved with Changes / Needs Clarification / Not Approved -->

Decision Date

<!-- Enter the date of the decision. -->

Comments or Conditions

<!-- Record comments, required changes, or approval conditions. -->